

查找技术

查找 (Searching) 是在数据集合中寻找满足特定条件的数据元素的过程。查找算法的效率直接影响程序的性能。

NOTE

查找算法的核心评价指标是平均查找长度 (ASL, Average Search Length)，即查找成功或失败时需要进行的关键字比较次数的期望值。

基本概念

查找表

查找表是由同一类型的数据元素（或记录）构成的集合。

静态查找表：只做查找操作，不改变表的内容

动态查找表：在查找过程中同时进行插入或删除操作

多关键字查找：数据可能存在多个关键字，查找操作需要指定查询条件

关键字

关键字是数据元素中某个数据项的值，用以标识一个数据元素。

主关键字：可以唯一标识一个记录的关键字

次关键字：不能唯一标识记录的关键字

平均查找长度 (ASL)

ASL 是衡量查找算法效率的主要指标：

$$ASL = \sum_{i=1}^n p_i \cdot c_i$$

其中：

- n ：表中记录个数
- p_i ：查找第 i 个记录的概率
- c_i ：找到第 i 个记录所需的比较次数

TIP

通常假设查找每个记录的概率相等，即 $p_i = \frac{1}{n}$ 。

静态查找

静态查找表在查找过程中不发生变化，适合使用以下算法。

顺序查找

顺序查找是最基本的查找方法，从表的一端开始，逐个将记录的关键字与给定值进行比较。

基本实现

```
int sequentialSearch(vector<int>& arr, int target) {
    for (int i = 0; i < arr.size(); ++i) {
        if (arr[i] == target) {
            return i; // 找到，返回索引
        }
    }
    return -1; // 未找到
}
```

时间复杂度:

- 最好情况: $O(1)$ (第一个元素就是目标)
- 最坏情况: $O(n)$ (最后一个元素或不存在)
- 平均情况: $O(n)$

ASL:

- 查找成功: $ASL = \frac{n+1}{2}$
- 查找失败: $ASL = n$

哨兵优化

通过在数组开头设置“哨兵”，可以减少每次循环中的边界检查。

```
int sequentialSearchWithSentinel(vector<int>& arr, int target) {
    int n = arr.size();
    arr.insert(arr.begin(), target); // 在开头插入哨兵

    int i = n;
    while (arr[i] != target) {
        --i;
    }

    arr.erase(arr.begin()); // 恢复原数组
    return i == 0 ? -1 : i - 1; // i=0表示未找到
}
```

NOTE

哨兵优化虽然减少了比较次数，但需要修改原数组或使用额外空间，实际应用中需权衡利弊。

折半查找（二分查找）

折半查找要求查找表必须是有序的，通过不断将查找区间缩小一半来提高效率。

基本实现

```
int binarySearch(vector<int>& arr, int target) {
    int left = 0, right = arr.size() - 1;

    while (left <= right) {
        int mid = left + (right - left) / 2; // 防止溢出

        if (arr[mid] == target) {
            return mid;
        } else if (arr[mid] < target) {
            left = mid + 1;
        } else {
            right = mid - 1;
        }
    }

    return -1; // 未找到
}
```

时间复杂度: $O(\log n)$

ASL: $ASL \approx \log_2(n+1) - 1$

CAUTION

计算 mid 时使用 $\text{left} + (\text{right} - \text{left}) / 2$ 而非 $(\text{left} + \text{right}) / 2$, 可以避免整数溢出。

二分查找的变体

查找第一个等于目标值的元素

```
int findFirstEqual(vector<int>& arr, int target) {
    int left = 0, right = arr.size() - 1;
    int result = -1;

    while (left <= right) {
        int mid = left + (right - left) / 2;

        if (arr[mid] == target) {
            result = mid;
            right = mid - 1; // 继续向左查找
        } else if (arr[mid] < target) {
            left = mid + 1;
        } else {
            right = mid - 1;
        }
    }

    return result;
}
```

查找最后一个等于目标值的元素

```
int findLastEqual(vector<int>& arr, int target) {
    int left = 0, right = arr.size() - 1;
    int result = -1;

    while (left <= right) {
        int mid = left + (right - left) / 2;

        if (arr[mid] == target) {
            result = mid;
            left = mid + 1; // 继续向右查找
        } else if (arr[mid] < target) {
            left = mid + 1;
        } else {
            right = mid - 1;
        }
    }

    return result;
}
```

查找第一个大于等于目标值的元素 (下界)

```
int lowerBound(vector<int>& arr, int target) {
    int left = 0, right = arr.size();

    while (left < right) {
        int mid = left + (right - left) / 2;

        if (arr[mid] < target) {
            left = mid + 1;
        }
    }
}
```

```

        } else {
            right = mid;
        }
    }

    return left; // 返回插入位置
}

```

查找第一个大于目标值的元素（上界）

```

int upperBound(vector<int>& arr, int target) {
    int left = 0, right = arr.size();

    while (left < right) {
        int mid = left + (right - left) / 2;

        if (arr[mid] <= target) {
            left = mid + 1;
        } else {
            right = mid;
        }
    }

    return left; // 返回插入位置
}

```

TIP

C++ STL 提供了 `lower_bound` 和 `upper_bound` 函数，可以直接使用。

二分查找的应用

二分答案

当问题的答案具有单调性时，可以通过二分查找来确定答案。

典型问题：

- 最小化最大值
- 最大化最小值
- 求满足条件的最小/最大值

解题框架：

```

bool check(int mid) {
    // 判断mid是否满足条件
}

int binarySearchAnswer(int left, int right) {
    while (left < right) {
        int mid = left + (right - left) / 2;

        if (check(mid)) {
            right = mid; // 答案在左半部分
        } else {
            left = mid + 1; // 答案在右半部分
        }
    }
}

```

```
    return left;
}
```

分块查找（索引顺序查找）

分块查找结合了顺序查找和折半查找的优点，将表分成若干块，块间有序，块内无序。

算法思想

1. 建立索引表，记录每块的最大关键字和起始位置
2. 在索引表中用折半查找确定目标所在的块
3. 在块内用顺序查找找到目标

性能分析

设表长为 n ，分为 b 块，每块 s 个元素 ($n = b \times s$):

- 索引表查找: $O(\log b)$ 或 $O(b)$
- 块内查找: $O(s)$
- 总 ASL: $O(\log b + s)$ 或 $O(b + s)$

当 $b = s = \sqrt{n}$ 时，ASL 最小，为 $O(\sqrt{n})$ 。

NOTE

分块查找适用于动态查找表，插入和删除操作比折半查找更方便。

插值查找

插值查找是折半查找的改进，根据目标值在有序表中的可能位置进行调整。

算法思想

将折半查找的中点计算公式：

$$mid = \frac{left + right}{2}$$

改为：

$$mid = left + \frac{target - arr[left]}{arr[right] - arr[left]} \cdot (right - left)$$

适用场景

- 表长较大
- 关键字分布比较均匀

时间复杂度：

- 平均情况: $O(\log \log n)$
- 最坏情况: $O(n)$ （分布极不均匀时）

CAUTION

插值查找在关键字分布不均匀时性能可能退化，需谨慎使用。

斐波那契查找

斐波那契查找利用斐波那契数列来分割查找区间，避免使用除法运算。

算法思想

使用斐波那契数列 $F[k]$ 来分割区间，使得：

- 左子区间长度: $F[k-1] - 1$
- 右子区间长度: $F[k-2] - 1$

特点

- 只使用加减法，不涉及乘除运算
- 在某些硬件平台上可能更快
- 实际应用较少

TIP

在实际开发中，折半查找已经足够高效且易于理解，推荐使用折半查找。

树形查找结构

树形查找结构利用树的层次组织数据，支持高效的动态查找、插入和删除操作。

二叉搜索树（BST）

二叉搜索树是一种特殊的二叉树，满足以下性质：

- 若左子树非空，则左子树上所有节点的值均小于根节点的值
- 若右子树非空，则右子树上所有节点的值均大于根节点的值
- 左右子树也分别是二叉搜索树

基本操作

查找

```
TreeNode* searchBST(TreeNode* root, int val) {
    if (!root || root->val == val) {
        return root;
    }

    if (val < root->val) {
        return searchBST(root->left, val);
    } else {
        return searchBST(root->right, val);
    }
}
```

时间复杂度：

- 最好情况： $O(\log n)$ （平衡树）
- 最坏情况： $O(n)$ （退化为链表）

插入

```
TreeNode* insertBST(TreeNode* root, int val) {
    if (!root) {
        return new TreeNode(val);
    }

    if (val < root->val) {
        root->left = insertBST(root->left, val);
    } else if (val > root->val) {
        root->right = insertBST(root->right, val);
    }

    return root;
}
```

删除

删除操作分三种情况：

1. 叶子节点：直接删除
2. 只有一个子节点：用子节点替代
3. 有两个子节点：用中序后继（右子树最小节点）替代

```
TreeNode* deleteBST(TreeNode* root, int val) {
    if (!root) return nullptr;

    if (val < root->val) {
        root->left = deleteBST(root->left, val);
    } else if (val > root->val) {
        root->right = deleteBST(root->right, val);
    } else {
        // 找到要删除的节点
        if (!root->left) {
            TreeNode* temp = root->right;
            delete root;
            return temp;
        } else if (!root->right) {
            TreeNode* temp = root->left;
            delete root;
            return temp;
        } else {
            // 有两个子节点，找中序后继
            TreeNode* successor = findMin(root->right);
            root->val = successor->val;
            root->right = deleteBST(root->right, successor->val);
        }
    }

    return root;
}

TreeNode* findMin(TreeNode* node) {
    while (node->left) {
        node = node->left;
    }
    return node;
}
```

性能分析

BST 的性能取决于树的形态：

- 最好情况：完全平衡，高度为 $O(\log n)$
- 最坏情况：退化为链表，高度为 $O(n)$

CAUTION

BST 在插入顺序不利时会严重退化，实际应用中应使用平衡二叉搜索树。

平衡二叉树（AVL 树）

AVL 树是一种自平衡的二叉搜索树，任意节点的左右子树高度差不超过 1。

平衡因子

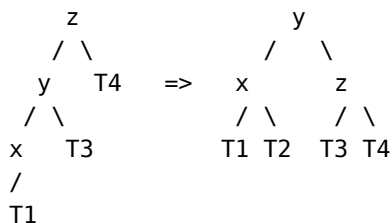
平衡因子 = 左子树高度 - 右子树高度

AVL 树要求所有节点的平衡因子为 -1、0 或 1。

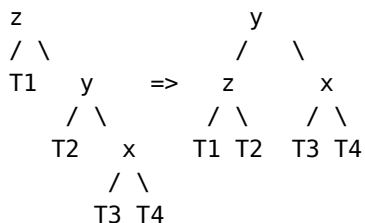
旋转操作

当插入或删除节点导致树不平衡时，需要通过旋转来恢复平衡。

LL 旋转（右旋）



RR 旋转（左旋）



LR 旋转（先左后右）

先对左子节点进行 RR 旋转，再对根节点进行 LL 旋转。

RL 旋转（先右后左）

先对右子节点进行 LL 旋转，再对根节点进行 RR 旋转。

性能分析

- 查找、插入、删除的时间复杂度均为 $O(\log n)$
- 保持严格平衡，查询效率高
- 插入和删除可能需要多次旋转

NOTE

AVL 树适合读多写少的场景，如数据库索引。

红黑树

红黑树是一种近似平衡的二叉搜索树，通过颜色约束来保证平衡。

红黑树的五条性质

1. 每个节点要么是红色，要么是黑色
2. 根节点是黑色
3. 每个叶子节点（NIL）是黑色
4. 如果一个节点是红色，则它的两个子节点都是黑色
5. 从任一节点到其每个叶子的所有简单路径都包含相同数目的黑色节点

性能特点

- 查找、插入、删除的时间复杂度均为 $O(\log n)$
- 插入和删除最多只需要常数级旋转

- 比 AVL 树更宽松，适合频繁插入删除的场景

TIP

C++ STL 中的 map 和 set、Java 中的 TreeMap 和 TreeSet 都基于红黑树实现。

B 树与 B+树

B 树和 B+树是多路平衡搜索树，专为磁盘存储设计。

B 树

m 阶 B 树的性质：

- 每个节点最多有 **m** 个子节点
- 除根节点外，每个节点至少有 $\lceil \frac{m}{2} \rceil$ 个子节点
- 所有叶子节点在同一层
- 节点中的关键字按升序排列

应用场景：文件系统、数据库索引

B+树

B+树是 B 树的变种，具有以下特点：

- 非叶子节点只存储索引，不存储数据
- 所有数据都存储在叶子节点
- 叶子节点之间用链表连接，便于范围查询

优势：

- 更适合范围查询
- 磁盘 I/O 次数更少
- 遍历效率更高

NOTE

MySQL 的 InnoDB 存储引擎使用 B+树作为索引结构。

Trie 树（前缀树）

Trie 树是一种用于字符串检索的多叉树结构。

结构特点

- 根节点不包含字符
- 每条边代表一个字符
- 从根到某节点的路径上的字符连接起来，即为该节点对应的字符串
- 每个节点的所有子节点包含的字符都不相同

基本操作

插入

```
struct TrieNode {
    TrieNode* children[26];
    bool isEnd;

    TrieNode() {
        for (int i = 0; i < 26; ++i) {
            children[i] = nullptr;
        }
    }
}
```

```

        isEnd = false;
    }
};

void insert(TrieNode* root, string word) {
    TrieNode* node = root;
    for (char c : word) {
        int idx = c - 'a';
        if (!node->children[idx]) {
            node->children[idx] = new TrieNode();
        }
        node = node->children[idx];
    }
    node->isEnd = true;
}

```

查找

```

bool search(TrieNode* root, string word) {
    TrieNode* node = root;
    for (char c : word) {
        int idx = c - 'a';
        if (!node->children[idx]) {
            return false;
        }
        node = node->children[idx];
    }
    return node->isEnd;
}

```

应用场景

- 自动补全
- 拼写检查
- IP 路由表查找
- 词频统计

时间复杂度: $O(m)$, 其中 m 是字符串长度

TIP

Trie 树的空间消耗较大, 可以使用压缩 Trie (基数树) 或双数组 Trie 来优化。

哈希表

哈希表通过哈希函数将关键字映射到表中的位置, 实现快速查找。

哈希函数构造方法

直接定址法

$$h(key) = key \text{ 或 } h(key) = a \cdot key + b$$

优点: 简单、无冲突

缺点: 只适用于关键字分布连续的情况

除留余数法

$$h(key) = key \bmod p, \text{ 其中 } p \leq m \text{ (表长)}$$

优点：最简单、最常用

注意：p 最好选择素数

数字分析法

抽取关键字中分布均匀的若干位作为哈希地址。

适用场景：关键字位数较多，且某些位分布均匀

平方取中法

取关键字平方后的中间几位作为哈希地址。

优点：适用于关键字分布不均匀的情况

折叠法

将关键字分割成位数相同的几部分，然后取叠加和作为哈希地址。

适用场景：关键字位数很多

随机数法

$$h(key) = random(key)$$

适用场景：关键字长度不等

TIP

除留余数法是最常用的哈希函数构造方法，实际应用中优先选择。

冲突处理方法

开放定址法

当发生冲突时，按照某种探测序列寻找下一个空闲位置。

线性探测法

$$h_i(key) = (h(key) + i) \bmod m, \quad i = 1, 2, \dots, m - 1$$

优点：实现简单

缺点：容易产生聚集现象

二次探测法

$$h_i(key) = (h(key) + i^2) \bmod m \text{ 或 } h_i(key) = (h(key) - i^2) \bmod m$$

优点：减少了一次聚集

缺点：仍然可能存在二次聚集

伪随机探测法

$$h_i(key) = (h(key) + pseudo_random(i)) \bmod m$$

优点：进一步减少聚集

链地址法（拉链法）

将所有哈希地址相同的记录链接成一个单链表。

优点：

- 不会产生聚集

- 删除操作简单
- 装填因子可以大于 1

缺点：

- 需要额外的指针空间
- 缓存友好性较差

NOTE

Java 的 `HashMap`、C++ 的 `unordered_map` 都使用链地址法处理冲突。

再哈希法

当发生冲突时，使用另一个哈希函数计算新的地址。

优点：不易产生聚集

缺点：增加计算时间

建立公共溢出区

将哈希表分为基本表和溢出表两部分，所有冲突的记录都放入溢出表。

性能分析

哈希表的性能取决于装填因子 α ：

$$\alpha = \frac{n}{m} = \frac{\text{表中记录数}}{\text{哈希表长度}}$$

查找成功的平均查找长度：

- 链地址法： $ASL \approx 1 + \frac{\alpha}{2}$
- 线性探测法： $ASL \approx \frac{1}{2}(1 + \frac{1}{1 - \alpha})$

查找失败的平均查找长度：

- 链地址法： $ASL \approx \alpha + e^{-\alpha}$
- 线性探测法： $ASL \approx \frac{1}{2}(1 + \frac{1}{(1 - \alpha)^2})$

CAUTION

装填因子越大，冲突概率越高，查找效率越低。一般建议 $\alpha \leq 0.75$ 。

哈希表的扩容与 Rehash

当装填因子超过阈值时，需要扩大哈希表容量并重新哈希所有元素。

扩容策略：

- 通常将表长扩大为原来的 2 倍
- 选择新的表长为素数
- 重新计算所有元素的哈希值并插入新表

时间复杂度： $O(n)$ ，但平摊到每次插入操作为 $O(1)$

工程应用

一致性哈希

一致性哈希用于分布式缓存系统，解决节点增减时的数据迁移问题。

核心理想：

- 将哈希空间组织成一个环
- 节点和数据都映射到环上
- 数据存储从顺时针方向的第一个节点上

优点：

- 节点增减时，只有少量数据需要迁移
- 负载均衡效果好

NOTE

Redis 集群、Memcached 等分布式缓存系统都使用一致性哈希。

布隆过滤器 (Bloom Filter)

布隆过滤器是一种空间高效的概率型数据结构，用于判断元素是否在集合中。

工作原理：

- 使用多个哈希函数将元素映射到位数组的多个位置
- 查询时，检查所有对应位置是否都为 1

特点：

- 可能存在误判（假阳性），但不会漏判（假阴性）
- 空间效率极高
- 不支持删除操作（可使用计数布隆过滤器解决）

应用场景：

- 网页去重
- 缓存穿透防护
- 垃圾邮件过滤

TIP

布隆过滤器的误判率可以通过调整位数组大小和哈希函数数量来控制。

数据库索引设计

哈希索引 vs B+树索引：

特性	哈希索引	B+树索引
等值查询	$O(1)$	$O(\log n)$
范围查询	不支持	支持
排序	不支持	支持
最左前缀	不支持	支持
空间占用	较小	较大

NOTE

MySQL 的 Memory 存储引擎支持哈希索引，InnoDB 支持自适应哈希索引。

查找算法比较与选择

各种查找算法对比

算法	平均时间复杂度	是否需要有序	适用场景
顺序查找	$O(n)$	否	小规模、无序数据
折半查找	$O(\log n)$	是	静态有序表
分块查找	$O(\sqrt{n})$	块间有序	动态查找表
BST	$O(\log n) \sim O(n)$	隐含有序	动态查找
AVL 树	$O(\log n)$	隐含有序	读多写少
红黑树	$O(\log n)$	隐含有序	频繁插入删除
B+树	$O(\log n)$	隐含有序	磁盘存储、数据库
哈希表	$O(1)$	否	等值查询
Trie 树	$O(m)$	否	字符串检索

选择策略

根据数据特点选择

- 数据量小：顺序查找即可
- 数据有序：折半查找
- 需要范围查询：B+树、平衡二叉树
- 只需等值查询：哈希表
- 字符串前缀匹配：Trie 树

根据操作频率选择

- 查多改少：AVL 树、B+树
- 查改频繁：红黑树、哈希表
- 动态插入删除：平衡二叉树、哈希表

根据存储介质选择

- 内存：哈希表、平衡二叉树
- 磁盘：B+树、B 树

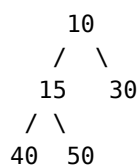
二叉堆

二叉堆（Binary Heap）是一种完全二叉树，同时满足特定的堆性质，广泛应用于优先队列和堆排序中。

堆的性质

完全二叉树

二叉堆是一棵完全二叉树，叶节点都尽可能靠近树的左侧。如果一棵树的高度为 h ，则除了最后一层，其他层都必须是满的；最后一层的节点从左到右依次排列。



堆序性

- 最大堆 (**Max Heap**) : 对于每个节点, 其值都大于或等于其子节点的值。根节点包含整个堆中的最大值。
- 最小堆 (**Min Heap**) : 对于每个节点, 其值都小于或等于其子节点的值。根节点包含整个堆中的最小值。

数组实现

二叉堆通常使用数组来表示, 利用索引访问节点的父节点和子节点。

假设堆使用数组 `arr[]` 表示, 节点编号从 0 开始:

- 父节点索引: $parent(i) = \frac{i-1}{2}$
- 左子节点索引: $left(i) = 2 \cdot i + 1$
- 右子节点索引: $right(i) = 2 \cdot i + 2$

NOTE

数组实现避免了显式的指针操作, 空间效率高, 缓存友好性好。

基本操作

插入操作 (上浮)

```
void insert(vector<int>& heap, int val) {
    heap.push_back(val);
    int i = heap.size() - 1;

    // 上浮调整
    while (i > 0) {
        int parent = (i - 1) / 2;
        if (heap[i] < heap[parent]) { // 最小堆
            swap(heap[i], heap[parent]);
            i = parent;
        } else {
            break;
        }
    }
}
```

时间复杂度: $O(\log n)$

删除堆顶 (下沉)

```
int extractMin(vector<int>& heap) {
    if (heap.empty()) throw runtime_error("Heap is empty");

    int minVal = heap[0];
    heap[0] = heap.back();
    heap.pop_back();

    // 下沉调整
    int i = 0;
    int n = heap.size();

    while (true) {
        int left = 2 * i + 1;
        int right = 2 * i + 2;
        int smallest = i;
```

```

        if (left < n && heap[left] < heap[smallest]) {
            smallest = left;
        }
        if (right < n && heap[right] < heap[smallest]) {
            smallest = right;
        }

        if (smallest != i) {
            swap(heap[i], heap[smallest]);
            i = smallest;
        } else {
            break;
        }
    }

    return minVal;
}

```

时间复杂度: $O(\log n)$

建堆操作

```

void buildHeap(vector<int>& arr) {
    int n = arr.size();

    // 从最后一个非叶子节点开始下沉
    for (int i = n / 2 - 1; i >= 0; --i) {
        heapify(arr, n, i);
    }
}

void heapify(vector<int>& arr, int n, int i) {
    int smallest = i;
    int left = 2 * i + 1;
    int right = 2 * i + 2;

    if (left < n && arr[left] < arr[smallest]) {
        smallest = left;
    }
    if (right < n && arr[right] < arr[smallest]) {
        smallest = right;
    }

    if (smallest != i) {
        swap(arr[i], arr[smallest]);
        heapify(arr, n, smallest);
    }
}

```

时间复杂度: $O(n)$ (比逐个插入的 $O(n \log n)$ 更优)

TIP

建堆操作的时间复杂度是 $O(n)$ 而非 $O(n \log n)$, 这是因为大部分节点位于树的底层, 下沉操作的代价很小。

其他平衡二叉搜索树

伸展树 (Splay Tree)

伸展树是一种自调整的二叉搜索树，通过 **splay** 操作将最近访问的节点移动到根部。

核心思想

- 局部性原理：最近访问的节点很可能再次被访问
- 每次访问节点后，通过旋转将其移动到根部
- 不需要存储额外的平衡信息

Splay 操作

splay 操作通过一系列旋转将目标节点移动到根部：

- **Zig**: 节点是根的左/右子节点，单次旋转
- **Zig-Zig**: 节点和父节点都是左/右子节点，两次同向旋转
- **Zig-Zag**: 节点和父节点分别是左/右子节点，两次反向旋转

性能特点

- 平摊时间复杂度: $O(\log n)$
- 无需额外空间: 不需要存储平衡因子或颜色
- 自适应: 频繁访问的节点靠近根部

NOTE

伸展树适合具有局部性特征的访问模式，如缓存系统。

Treap (树堆)

Treap 是 Tree 和 Heap 的组合，每个节点除了关键字外还有一个随机优先级，同时满足 BST 性质和堆性质。

性质

- **BST 性质**: 按关键字排序
- **堆性质**: 按优先级排序 (通常是最大堆)
- **随机性**: 优先级随机生成，保证树的期望高度为 $O(\log n)$

基本操作

插入

1. 按 BST 规则插入新节点，赋予随机优先级
2. 通过旋转维护堆性质

删除

1. 如果要删除的节点是叶子，直接删除
2. 否则，通过旋转将其下降到叶子位置，然后删除

优势

- 实现简单: 相比 AVL 和红黑树更容易实现
- 期望平衡: 通过随机化保证期望性能
- 支持分裂合并: 可以高效地分裂和合并 Treap

TIP

Treap 的随机化策略使其在实践中表现良好，且代码简洁。

跳表 (Skip List)

跳表是一种多级索引的有序链表数据结构，可用于快速查找。

结构特点

- 多层链表，每层是下一层的子集
- 最底层包含所有元素
- 高层作为“快速通道”，跳过大量元素

```
Level 3:    1 -----> 9
Level 2:    1 -----> 5 -----> 9
Level 1:    1 --> 3 --> 5 --> 7 --> 9
Level 0:    1 -> 2 -> 3 -> 4 -> 5 -> 6 -> 7 -> 8 -> 9
```

查找过程

1. 从最高层开始向右查找
2. 如果下一个节点大于目标，下降到下一层
3. 重复直到找到目标或到达最底层

时间复杂度：

- 平均情况： $O(\log n)$
- 最坏情况： $O(n)$ （概率极低）

插入操作

1. 查找到插入位置
2. 随机决定新节点的层数
3. 更新各层的指针

优势

- 实现简单：比平衡树更容易实现
- 并发友好：易于实现无锁并发
- 动态性好：插入删除操作简单

NOTE

Redis 的有序集合 (Sorted Set) 底层使用跳表实现。

工程应用与优化

数据库索引设计

B+树 vs LSM 树

特性	B+树	LSM 树
读性能	优秀	一般
写性能	一般	优秀
范围查询	优秀	良好
空间放大	较小	较大
写放大	较小	较大
典型应用	MySQL InnoDB	LevelDB, RocksDB

B+树：

- 适合读多写少的场景
- 支持高效的范围查询
- 磁盘 I/O 次数少

LSM 树 (Log-Structured Merge Tree) :

- 适合写密集型应用
- 通过批量合并减少随机写
- 读操作可能需要合并多个文件

TIP

选择索引结构时需要根据应用的读写比例进行权衡。

倒排索引

倒排索引是搜索引擎的核心数据结构，建立关键词到文档的映射。

结构

关键词：[文档ID列表]

"算法": [1, 3, 5, 7]

"数据结构": [2, 3, 6, 8]

"查找": [1, 2, 4, 5]

构建过程

1. 分词：将文档分解为关键词
2. 建立映射：记录每个关键词出现的文档
3. 排序：对文档 ID 列表排序，便于合并

查询优化

- 布尔查询：AND、OR、NOT 操作
- 短语查询：检查关键词的位置关系
- 相关性排序：TF-IDF、BM25 等算法

NOTE

Elasticsearch、Lucene 等搜索引擎都使用倒排索引。

缓存优化

LRU (Least Recently Used)

淘汰最近最少使用的数据。

实现：哈希表 + 双向链表

- 哈希表： $O(1)$ 查找
- 双向链表：维护访问顺序

```
class LRUCache {
private:
    int capacity;
    list<pair<int, int>> cache; // key, value
    unordered_map<int, list<pair<int, int>>::iterator> map;

public:
    LRUCache(int cap) : capacity(cap) {}
}
```

```

int get(int key) {
    if (map.find(key) == map.end()) return -1;

    // 移动到链表头部
    cache.splice(cache.begin(), cache, map[key]);
    return map[key]->second;
}

void put(int key, int value) {
    if (map.find(key) != map.end()) {
        cache.erase(map[key]);
    } else if (cache.size() == capacity) {
        // 淘汰尾部
        map.erase(cache.back().first);
        cache.pop_back();
    }

    cache.emplace_front(key, value);
    map[key] = cache.begin();
}
};

```

时间复杂度： $O(1)$

LFU (Least Frequently Used)

淘汰使用频率最低的数据。

实现：

- 哈希表：key 到节点的映射
- 频率桶：每个频率对应一个双向链表
- 最小频率追踪

应用场景：

- 缓存热点数据
- 数据库缓冲池

CAUTION

LFU 比 LRU 更复杂，但在某些场景下效果更好，特别是当访问模式具有明显的热点特征时。

搜索算法优化

搜索算法用于在解空间中寻找满足条件的解，常用于组合优化问题。

剪枝技术

剪枝通过排除不可能的分支来减少搜索空间。

可行性剪枝

直接排除不满足约束条件的分支。

示例：N 皇后问题中，如果当前位置与已放置的皇后冲突，则不再继续搜索。

最优性剪枝

如果当前路径的代价已经劣于已知最优解，则剪枝。

示例：旅行商问题中，如果当前路径长度已经超过已知最短路径，则停止搜索。

```

void dfs(int node, int cost, int visited) {
    // 最优性剪枝
    if (cost >= bestCost) return;

    if (visited == allVisited) {
        bestCost = min(bestCost, cost + dist[node][start]);
        return;
    }

    for (int next = 0; next < n; ++next) {
        if (!(visited & (1 << next))) {
            dfs(next, cost + dist[node][next], visited | (1 << next));
        }
    }
}

```

α - β 剪枝

用于博弈树搜索，通过维护上下界来剪枝。

核心理想：

- α : MAX 节点的下界
- β : MIN 节点的上界
- 如果 $\alpha \geq \beta$ ，则剪枝

应用场景：棋类游戏 AI（国际象棋、围棋等）

TIP

α - β 剪枝可以将搜索效率提高一倍以上，是博弈树搜索的标准优化技术。

双向 BFS

双向 BFS 从起点和终点同时开始搜索，当两个搜索相遇时找到最短路径。

算法流程

1. 初始化两个队列，分别从起点和终点开始
2. 交替扩展两个队列
3. 当某个节点被两个方向都访问到时，找到最短路径

时间复杂度优化

- 单向 BFS: $O(b^d)$ ，其中 b 是分支因子， d 是深度
- 双向 BFS: $O(b^{\{\frac{d}{2}\}})$ ，指数级优化

```

int bidirectionalBFS(int start, int end) {
    if (start == end) return 0;

    queue<int> q1, q2;
    unordered_set<int> visited1, visited2;

    q1.push(start); visited1.insert(start);
    q2.push(end); visited2.insert(end);

    int steps = 0;

    while (!q1.empty() && !q2.empty()) {
        // 扩展较小的队列

```

```

        if (q1.size() > q2.size()) {
            swap(q1, q2);
            swap(visited1, visited2);
        }

        int size = q1.size();
        for (int i = 0; i < size; ++i) {
            int node = q1.front(); q1.pop();

            if (visited2.count(node)) {
                return steps;
            }

            for (int neighbor : getNeighbors(node)) {
                if (!visited1.count(neighbor)) {
                    visited1.insert(neighbor);
                    q1.push(neighbor);
                }
            }
            ++steps;
        }

        return -1; // 未找到路径
    }
}

```

NOTE

双向 BFS 适用于无向图或双向可达的有向图，且需要能够快速判断节点是否被另一个方向访问过。

记忆化搜索

记忆化搜索是自顶向下的动态规划，通过缓存子问题的解避免重复计算。

与 DP 的关系

- 记忆化搜索：递归 + 缓存，自顶向下
- 动态规划：迭代 + 表格，自底向上
- 两者本质相同，只是计算顺序不同

经典示例

斐波那契数列

```

unordered_map<int, int> memo;

int fib(int n) {
    if (n <= 1) return n;
    if (memo.count(n)) return memo[n];
    return memo[n] = fib(n - 1) + fib(n - 2);
}

```

时间复杂度：从 $O(2^n)$ 优化到 $O(n)$

最长公共子序列

```

vector<vector<int>> memo;

int lcs(string& s1, string& s2, int i, int j) {
    if (i == 0 || j == 0) return 0;
}

```

```

    if (memo[i][j] != -1) return memo[i][j];

    if (s1[i - 1] == s2[j - 1]) {
        return memo[i][j] = 1 + lcs(s1, s2, i - 1, j - 1);
    } else {
        return memo[i][j] = max(lcs(s1, s2, i - 1, j), lcs(s1, s2, i, j - 1));
    }
}

```

TIP

记忆化搜索的优点是只计算需要的状态，而 DP 会计算所有状态。对于状态空间稀疏的问题，记忆化搜索更高效。

迭代加深搜索 (IDS)

迭代加深搜索结合了 DFS 的空间效率和 BFS 的最优性。

算法思想

1. 从深度限制 $d = 1$ 开始
2. 执行深度限制的 DFS
3. 如果未找到解，增加深度限制 $d++$
4. 重复直到找到解

适用场景

- 深度未知或很大的搜索树
- 需要找到最优解（最短路径）
- 空间受限，无法使用 BFS

时间复杂度

虽然会重复搜索浅层节点，但总时间复杂度与 BFS 相同： $O(b^d)$

原因：大部分时间在搜索最深层，浅层的重复开销相对较小。

```

bool DLS(int node, int target, int depth) {
    if (node == target) return true;
    if (depth <= 0) return false;

    for (int neighbor : getNeighbors(node)) {
        if (DLS(neighbor, target, depth - 1)) {
            return true;
        }
    }
    return false;
}

int IDS(int start, int target) {
    for (int depth = 0; ; ++depth) {
        if (DLS(start, target, depth)) {
            return depth;
        }
    }
}

```

NOTE

IDS 在实际应用中非常有用，特别是在游戏 AI 和谜题求解中。

启发式搜索

启发式搜索利用启发函数指导搜索方向，提高效率。

A*算法

A*算法是最著名的启发式搜索算法，结合了实际代价和启发估计。

评估函数：

$$f(n) = g(n) + h(n)$$

- $g(n)$ ：从起点到节点 n 的实际代价
- $h(n)$ ：从节点 n 到终点的启发估计

最优性条件：

- 如果 $h(n) \leq h^*(n)$ （不高估真实代价），A*保证找到最优解
- $h(n) = 0$ 时，A*退化为 Dijkstra 算法
- $h(n)$ 越接近真实值，搜索效率越高

实现：

```
struct Node {
    int id;
    double g, h, f;
    bool operator>(const Node& other) const {
        return f > other.f;
    }
};

vector<int> astar(int start, int end,
                 const vector<vector<pair<int, double>>>& graph,
                 function<double(int, int)> heuristic) {
    priority_queue<Node, vector<Node>, greater<Node>> pq;
    unordered_map<int, double> gScore;
    unordered_map<int, int> cameFrom;

    pq.push({start, 0, heuristic(start, end), 0});
    gScore[start] = 0;

    while (!pq.empty()) {
        Node current = pq.top(); pq.pop();

        if (current.id == end) {
            // 重构路径
            vector<int> path;
            for (int at = end; at != start; at = cameFrom[at]) {
                path.push_back(at);
            }
            path.push_back(start);
            reverse(path.begin(), path.end());
            return path;
        }

        if (current.g > gScore[current.id]) continue;

        for (auto& [neighbor, weight] : graph[current.id]) {
            double tentativeG = current.g + weight;
```



```

        if (!gScore.count(neighbor) || tentativeG < gScore[neighbor]) {
            gScore[neighbor] = tentativeG;
            double h = heuristic(neighbor, end);
            pq.push({neighbor, tentativeG, h, tentativeG + h});
            cameFrom[neighbor] = current.id;
        }
    }
}

return {}; // 未找到路径
}

```

启发函数设计

常见启发函数：

- 曼哈顿距离：适用于网格地图，只能上下左右移动
- 欧几里得距离：适用于可以任意方向移动的场景
- 对角线距离：适用于可以斜向移动的网格

设计原则：

- 可采纳性：不高估真实代价（保证最优性）
- 一致性： $h(n) \leq c(n, n') + h(n')$ （保证效率）

应用对比

算法	启发函数	最优性	适用场景
Dijkstra	$h(n)=0$	是	无权图或多目标
A*	$h(n) \leq h^*(n)$	是	单源单目标最短路径
贪婪最佳优先	$h(n)$	否	快速找到可行解

TIP

A*算法的性能很大程度上取决于启发函数的质量。好的启发函数可以大幅减少搜索节点数。

本章完